IDfRA: Self-Verification for Iterative Design in Robotic Assembly

Nishka Khendry[1] and Christos Margadji[1] and Sebastian W. Pattinson[1]

arXiv:2509.16998v1 [cs.RO] 21 Sep 2025

***Abstract*— As robots proliferate in manufacturing, Design for Robotic Assembly (DfRA), which is designing products for efficient automated assembly, is increasingly important. Traditional approaches to DfRA rely on manual planning, which is time-consuming, expensive and potentially impractical for complex objects. Large language models (LLM) have exhibited proficiency in semantic interpretation and robotic task planning, stimulating interest in their application to the automation of DfRA. But existing methodologies typically rely on heuristic strategies and rigid, hard-coded physics simulators that may not translate into real-world assembly contexts. In this work, we present Iterative Design for Robotic Assembly (IDfRA), a framework using iterative cycles of planning, execution, verification, and re-planning, each informed by self-assessment, to progressively enhance design quality within a fixed yet initially under-specified environment, thereby eliminating the physics simulation with the real world itself. The framework accepts as input a target structure together with a partial environmental representation. Through successive refinement, it converges toward solutions that reconcile semantic fidelity with physical feasibility. Empirical evaluation demonstrates that IDfRA attains 73.3% top-1 accuracy in semantic recognisability, surpassing the baseline on this metric. Moreover, the resulting assembly plans exhibit robust physical feasibility, achieving an overall 86.9% construction success rate, with design quality improving across iterations, albeit not always monotonically. Pairwise human evaluation further corroborates the advantages of IDfRA relative to alternative approaches. By integrating self-verification with context-aware adaptation, the framework evidences strong potential for deployment in unstructured manufacturing scenarios.**

I. Introduction

Robotic assembly involved using robots in manufacturing to automatically assemble components into finished products. While traditionally limited to repetitive tasks for industrial efficiency [1], manufacturing is shifting toward flexible, adaptive systems that operate in dynamic, unstructured environments [2], [3]. This transition has driven interest in Design for Robotic Assembly (DfRA), which involves concurrent design of both products and the robotic systems that assemble them [4]. However, conventional DfRA remains largely manual and template-based [5]–[7], constrained by vast design spaces, reliance on experience-based heuristics, and time-intensive tasks [8] – often resulting in suboptimal outcomes. This motivates automated approaches to systematically explore and optimise design alternatives.

Most prior work in automated DfRA focuses on Assembly Sequence Planning (ASP), a subcomponent of DfRA which optimises the order of pre-specified assembly steps. Notable examples include a transformer-based model that derives assembly sequences from a target blueprint [9], and a constraint-based tree search to generate feasible sequences automatically [10]. However, such methods address only a subset of DfRA and lack adaptability to dynamic contexts.

In parallel, large language and vision-language models (LLMs and VLMs) like GPT-4o have shown strong capabilities in semantic reasoning and high-level task planning [11]–[16], including promising results in diverse manufacturing simulations [17]. Leveraging this, [18] proposed the first LLM-driven automated DfRA system. However, their method selects one from multiple candidate designs generated in parallel, which precludes dynamic, incremental enhancement of design features or learning from past mistakes.

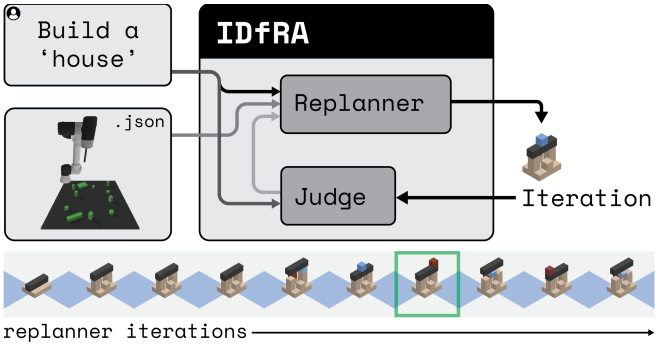

Fig. 1. The overall IDfRA framework. Top: Given a target assembly (e.g., "house") and a signature of the environment in .json format, IDfRA iteratively generates a detailed assembly plan specifying each available block's position and pose. **Bottom**: Renders of the 10 iterations of the "house" assembly.

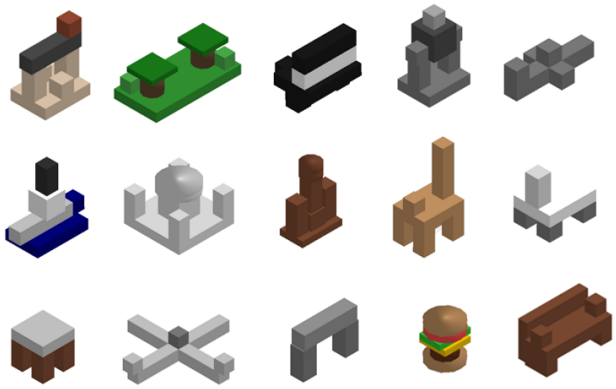

Fig. 2. 15 representative assemblies generated by IDfRA. Top left to bottom right: house, jungle, piano, robot, shark, ship, Taj Mahal, Eiffel Tower, giraffe, sheep, dining table set, ceiling fan, bridge, burger, couch.

Codebase: https://github.com/cam-cambridge/open-ended-assembly

This work was supported by the Engineering and Physical Sciences Research Council (EPSRC) of the United Kingdom

[1]Institute for Manufacturing, Department of Engineering, University of Cambridge {nk680,cm2161,swp29}@cam.ac.uk

As manufacturing shifts toward flexibility, there is a need for DfRA systems that iteratively improve through feedback. Such adaptability enables learning from previous mistakes and context-aware design refinement, which are crucial in variable industrial environments [19]. To address this, we introduce IDfRA, a framework for Iterative DfRA, which integrates self-verification and re-planning to enable such adaptability; this creates scope to optimally accommodate changing assembly contexts, such as variations in available components, site conditions, or manufacturing tolerances.

IDfRA employs self-verification – a robotic system's ability to assess its own actions and outcomes – as its feedback mechanism. Self-assessment is vital in uncertain or safety-critical environments [20], and has been shown to improve performance by enabling iterative refinement of task plans in both robotics [21]–[23] and other domains [24]. IDfRA addresses three core needs in modern DfRA: *adaptability to dynamic contexts* via iterative refinement, *safety* through embedded self-assessment, and *efficiency* via end-to-end automation. The contributions of this work are:

- IDfRA, which to our knowledge, is the first fully automated framework combining VLM-based self-verification with LLM-based re-planning for iterative design improvement.
- A systematic evaluation showing that IDfRA generates semantically meaningful, physically feasible designs that evolve through feedback, providing insights into foundation model reasoning within DfRA.

II. Related work

A. Robotic Assembly

Robotic systems are now widely integrated into industrial manufacturing [25], particularly in automotive and aerospace, where they have significantly enhanced production efficiency [2]. Consequently, Design for Robotic Assembly (DfRA) has emerged as a foundational concept.

Early methods for Design for Assembly (later extended to robotics) optimised assembly processes to minimise cost [26]–[28] via rule-based and manual strategies. Their limitations in quantifying exectuability motivated a shift towards physics-based reasoning, which catalysed widespread adoption of Computer-Aided Design (CAD) and Computer-Aided Manufacturing (CAM) software for visualisation and manual evaluation of components and assemblies [29]–[31]. Contemporary CAD and simulation-based tools, integrated with updated DfRA frameworks derived from the original models [26]–[28] remain the industry standard. Although these approaches advance partial automation of DfRA, they still necessitate close collaboration with engineers throughout the process. Manual DfRA is hindered by a vast solution space, which often leads to suboptimal designs and heavy reliance on designers' procedural knowledge and intuition [8]. Thus, greater automation could significantly improve efficiency and design quality.

Recent shifts in manufacturing towards flexibility and adaptability [32] have amplified interest in automating diverse assembly tasks in unstructured environments [2], [3]. Combined with the limitations of manual methods, this has driven application of ML to robotic assembly – primarily to ASP while DfRA remains largely manual. For example, much ASP research [10], [33]–[36] builds on Assembly-by-Disaembly (AbD), which analyses disassembly steps to inform assembly plans [37]. Meanwhile, LLMs and VLMs have been employed to decode IKEA instruction manuals [38] and engineering documentation [39], manipulate CAD sketches [40], decompose high-level construction tasks [41], and generate robot code from assembly specifications [42]. These methods demonstrate the significant potential of large pre-trained models in robotic assembly. However, prior ASP work focuses on sequencing predefined designs rather than generating new, physically valid assemblies from scratch.

BloxNet [18] directly tackles this by introducing the first fully automated DfRA system combining VLMs and physics simulation to generate 10 candidate designs and select the most semantically recognisable one. While effective at producing diverse structures from textual prompts, it has key limitations. First, it is constrained by its reliance on a physics engine for stability correction, which may not accurately extend to modern industrial environments. This is due to factors such as world modelling inaccuracies and significant simulation-to-real gaps [43], particularly in robotic assembly [44]. Recorded execution visuals are therefore more reliable than simulation outcomes to refine plans in physical assembly systems. Second, verification is minimal, with only three iterations focused solely on stability adjustment rather than overall design evaluation. Third and crucially, each candidate design is generated independently in parallel, preventing learning across iterations based on the current context and past mistakes (Fig 4b). In contrast, IDfRA leverages LLMs and VLMs within an iterative self-verification loop to dynamically refine design features with the goal of enabling more consistent and context-aware design refinement.

B. Robotic Planning and Self-Verification with Large Pre-Trained Models

LLMs offer strong potential for robotic planning due to their broad semantic understanding and creativity [45], Yet, they often fail to generate feasible plans in a single pass, and perform far better when combined with external verification and feedback [46]. This insight has driven a surge of research into iterative planning pipelines.

Several works employ execution-derived signals, such as simulator outputs or code errors, as sources of feedback [22], [24], [47]. In contrast, others use language-based correction or guidance to enhance iterative planning such as via knowledge bases of compliant plans [48] and predefined task guidelines [49]. Furthermore, [23] demonstrates the value of combining diverse feedback sources, such as success detection and scene description, and reports significant improvement in task completion across multiple robot environments. Collectively, these studies demonstrate the value of feedback-driven refinement and the promise of LLMs/VLMs for iterative assembly design.

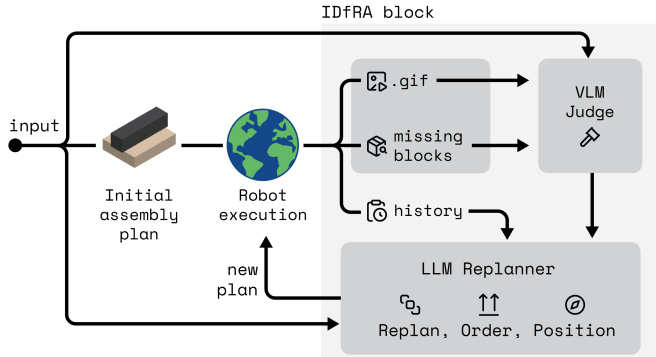

Fig. 3. IDfRA system architecture. The IDfRA framework iteratively refines assembly designs given a user-specified target structure and available blocks. Initially, a minimal design is constructed by the simulated robotic system . A GIF of this attempt – along with a list of any missing blocks used in the plan and the original system inputs – is passed into the *VLM Judge* module, implemented as a single GPT-4o instance. Based on the *Judge*'s feedback and previous plans, the *Replanner* module, comprising three GPT-4o instances – *Replan*, *Order*, and *Position* – produces an updated assembly design. The revised plan is then executed by the simulated robot and this process repeats for 10 iterations. After completion, any designs that require unavailable blocks are discarded and a *Selector* module, consisting of a single GPT-4o instance, selects the final assembly design.

III. Methods

A. Task Definition

We adopt the Generative DfRA task formulation introduced by BloxNet [18]. Overall system inputs include a target structure name (e.g., "house") and a JSON-formatted list of available blocks (including their dimensions, quantities, and shapes: cuboidal or cylindrical). The goal is to generate a complete assembly plan, specifying for each block its shape, semantic name, colour, 3D centre coordinate (X, Y, Z), and yaw angle.

B. System Architecture

While we adopt the same task definition as BloxNet, our approach addresses its key limitations (Section II-A). Design generation guided by three objectives: (1) semantic resemblance to the target, (2) physical feasibility for construction by the simulated robot, and (3) iterative enhancement of design features via feedback. The overall architecture and execution flow of IDfRA is detailed in Fig 3. The process begins with a single candidate design generated by BloxNet, with physics-based stability correction disabled to allow refinement of potentially unstable configurations. Object tracking is handled by PyBullet, while missing blocks are identified by comparing the assembly plan against the available set using Python. Assuming these functions can be replicated in physical systems via vision-based localisation is reasonable, given recent advancements in generalist agents capable of accurate object tracking and pick-and-place [50], [51]. The IDfRA framework consists of four key components:

1) VLM-Based Self-Verification Module (Judge): The *Judge*, implemented as a GPT-4o instance, is queried with a GIF of the robot's latest assembly attempt and a structured prompt incorporating: (1) the target structure name, (2) a JSON object specifying available blocks, (3) a list of missing blocks, and (4) the current JSON-formatted assembly plan (with LLM-generated semantic block names stripped). This setup ensures the *Judge* relies solely on visual input for semantic inference, thereby fostering independent understanding and unbiased feedback. To further support accurate visual reasoning, unused blocks are removed from the scene during execution.

The Judge analyses the GIF and infers the semantic role of each block (e.g., identifying a small block on a "roof" as a "chimney"). Based on this analysis, it returns a structured JSON assessment including: (1) identification of unavailable blocks and their associated design features, including quantity mismatches and instructions for the Replanner to operate within constraints, (2) evaluation of whether any missing blocks at intermediate stages may affect overall stability, (3) assessment of the semantic resemblance to the target structure, (4) suggestions for design improvement such as (a) adjustments to proportions or positioning of existing features or (b) recommendations for addition or removal of features. Explicit block availability analysis was included in the prompt to mitigate GPT-4o's tendency to hallucinate unavailable components – an issue also noted by [18].

A system prompt defines the *Judge*'s role as a "precise and critical evaluator" of robotic assemblies, while a user prompt provides updated context for each iteration. Full prompts are available in the released codebase. Semantic scene description was prioritised over topple detection as the primary feedback signal, given empirical evidence that current VLMs are stronger at visual scene understanding than multi-object tracking [52].

2) LLM-Based Re-Planning Module (Replanner): The *Replanner* comprises three GPT-4o instances arranged in a tiered structure, inspired by prior work [18], [22], to creatively incorporate feedback and progressively increase precision to generate a feasible assembly plan with explicit coordinates. The *Replanner* takes in the overall system inputs, latest *Judge* feedback, and a JSON record of all previous plans. Its output is a new JSON plan specifying each block's dimensions, 3D centre position, and yaw angle, which is then executed by the simulated robot (Section III-B.3).

Internally, the *Replanner* comprises three submodules, each fulfilling a distinct objective. First, *Replan* processes all inputs and incorporates *Judge* feedback to generate a high-level plan from scratch – specifying each block's colour, dimensions, semantic name, and relative placement. The colours generated are displayed in the rendered assembly images, while all blocks in the simulation remain green to mimic real conditions in the *Judge* input. Next, *Order* corrects illogical sequencing (such as placing upper components before foundational ones) to produce a feasible bottom-up assembly. Finally, *Position* generates explicit 3D centre coordinates (X, Y, Z) and yaw angles.

Prompt engineering was critical, especially for *Posi-*

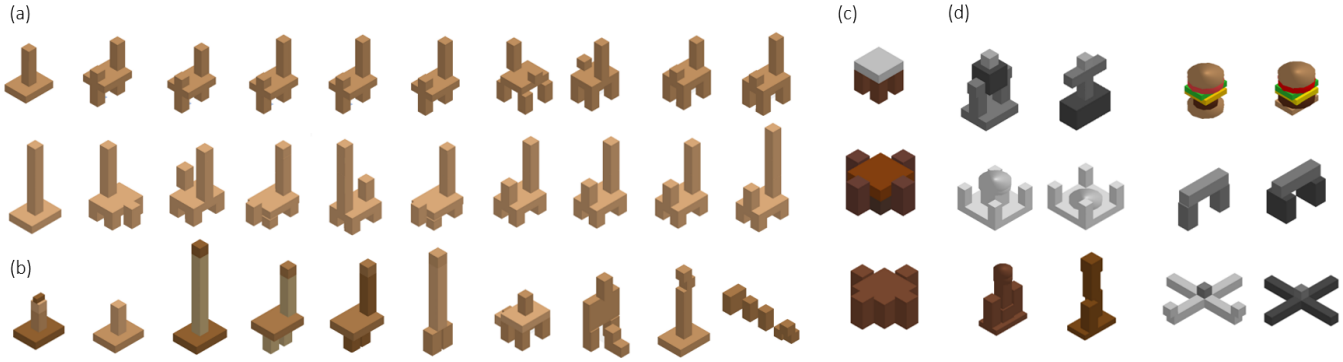

Fig. 4. IDfRA design generation. (a) Multiple runs of IDfRA – each comprising 10 iterations – produce different design evolutions given the same blocks and input "giraffe". (b) 10 candidate designs produced in parallel by BloxNet – each design is produced independently without influence from the previous one. (c) Designs generated by multiple full runs of IDfRA for "dining table set". (d) Designs generated by IDfRA (left) and BloxNet (right) for "robot", "Taj Mahal", "Eiffel Tower", "burger", "ceiling fan", "bridge".

tion, where ambiguity reduction, negative examples, and adjacency-based placement rules were used to minimise overhangs and improve stability. *Replan* simplifies this step by allowing block dimension switching for reorientation, as in [18], enabling *Position* to focus on spatial layout. This stage required careful prompt engineering to minimise overhangs and enhance structural stability – employing strategies such as removing ambiguity, adding negative examples, and providing detailed instructions that guide positioning based on adjacent blocks. *Replan* simplifies this step by allowing block dimension switching for reorientation as in [18], so that *Position* can focus solely on generating 3D centre coordinates.

Unlike the *Judge*, which uses a single system–user prompt pair, each *Replanner* submodule is driven by a distinct user prompt. This separation, supported by empirical results, improves performance by tailoring prompts to each subtask: *Replan* adapts flexibly to iteration-specific feedback, while *Order* and *Position* build directly on *Replan*'s output.

3) Robot Assembly Execution Module: Generated assembly plans are executed using a simulated robotic arm in a tabletop environment. For blocks requiring dimension switching, the corresponding roll and pitch are calculated programmatically. For example, if the Y and Z axes are swapped, a roll of 90° on an available cuboid produces the desired block. Each placement is a pick-and-place action: the arm retrieves the block from its tracked location, and resolves any roll or pitch offsets prior to final placement. Only 90° roll and pitch offsets are used, since ±90° produce the same displacement and symmetric cuboids remain indistinguishable under such rotations. To avoid destabilising any partially built assembly, rotated blocks are first moved aside, reoriented, then re-grasped from the new top face for placement. In contrast, blocks without dimension switches are placed directly from their initialised poses. All motions use PyBullet's built-in functions: joint angles are computed via inverse kinematics and applied through position control with low gains to ensure slow, smooth movements. A custom suction gripper is used to minimise interference with nearby blocks during stacking.

The simulation serves as a proxy for a real robotic system, and GIFs of assembly attempts are intended to mirror real-world execution videos – as a means to establish proof of concept that such a method could be extended to diverse manufacturing environments.

4) VLM-Based Selector Module: While designs generally improve over iterations, earlier and simpler configurations may sometimes offer better stability or recognisability (Fig 1, Bottom). After ten iterations, images are rendered for each assembly by spawning blocks at the generated poses under gravity and rigid body dynamics. Plans containing any missing blocks are disqualified and the remaining undergo a pairwise knockout process. In each pair, a GPT-4o instance selects the more stable and semantically accurate design – automatically favouring stable structures, and among them, the one that better resembles the target. This continues until a single final design is selected.

C. Implementation Details

The task is executed in a simulated tabletop environment using a Universal Robot UR5e arm and custom suction gripper, both instantiated and controlled using PyBullet. Simulation parameters include: uniform object density 1000 $\mathrm{kg/m^3}$, lateral friction coefficient 0.5, spinning friction coefficient 0.2, gravity -9.81 $\mathrm{m/s^2}$. At each reset, block positions are randomised outside a configurable central region to reserve space for assembly, with minimum spacing to prevent overlap.

All GPT-4o instances are accessed via black box queries through the OpenAI API. JSON is used throughout the system when interfacing with OpenAI's API, as it is the de facto standard for structured data exchange and explicitly recommended by OpenAI [53]. Images and GIFs were encoded as base 64 strings in API messages. Temperatures were set per module based on the required balance of creativity and consistency: 0.4 (*Judge*), 0.5 (*Replan*, *Order*), 0.25 (*Position*), and 0 (*Selector*). All GPT-4o interactions and robot assembly attempts are logged for interpretability.

IV. Experiments

We evaluate IDfRA on the three objectives defined in Section III-B: semantic recognisability, physical feasibility, and

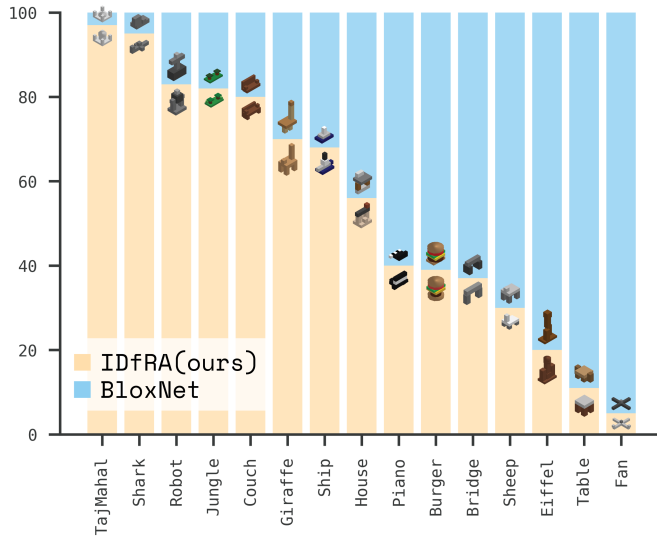

Fig. 5. Human evaluation of final designs. Pairwise voting scores from 100 survey responses comparing IDfRA vs BloxNet across 15 assemblies (best viewed zoomed-in.)

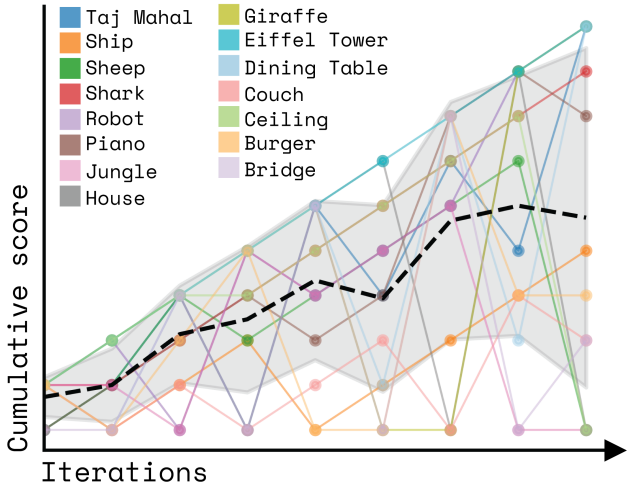

Fig. 6. Iterative design improvement. For each structure, the Y-axis shows the number of times an iteration matched or outperformed previous iterations. The mean trend is indicated by a black dashed line with standard deviation shaded in grey.

iterative improvement. A representative set of 15 assembly designs was selected to balance structural complexity and tractability: (1) each target structure is sufficiently intricate to support iterative refinement and to incorporate multiple components to achieve semantic resemblance, yet simple enough to be approximated using a limited block set, (2) the structures span varying levels of complexity to illustrate system flexibility.

For each target, a custom block set was prepared to: (1) provide the minimum blocks required for the target structure, (2) include surplus block types to allow creative variation rather than forcing a single design, and (3) remain within the tabletop workspace limits to support physical mappability. All JSON-formatted block sets (including dimensions and quantities) are available in the codebase.

Due to the stochastic nature of autoregressive decoding in VLMs, multiple IDfRA runs produce different design evolutions (Fig. 4a), some outperforming others (Fig. 4c). This variability also highlights IDfRA's ability to adaptively refine plans based on context and feedback. To ensure fair comparability with baseline BloxNet [18], and given the absence of detailed information regarding the exact block sets and number of runs used in its reported results, we ran both methods once using identical inputs (i.e., the target structure name and block set). Each method automatically generated its best designs, and we conducted two quantitative analyses to directly compare their performance. In addition, two analyses were conducted independently on IDfRA's outputs to assess its iterative refinement behaviour and physical feasibility of the generated assemblies.

A. Human Evaluation Survey

We conducted a human evaluation survey in which participants compared each pair of designs (IDfRA vs. BloxNet) for the 15 representative assemblies, and selected the one that more closely resembled the target structure. Order within each pair was randomised to mitigate bias. Participants were instructed to consider overall shape and key features, block usage, and stability. Recruitment combined convenience and snowball sampling.

Responses from 100 participants (Fig. 5) show that IDfRA was preferred in 8 out of 15 designs, achieving a win rate of 54.2% (813/1,500 pairwise comparisons). IDfRA outperformed BloxNet on more complex structures with multiple components such as "robot" and "Taj Mahal" (Fig 4d), while BloxNet performed better on simpler targets such as "ceiling fan" and "bridge". In cases like "Eiffel Tower" and "burger", participants favoured BloxNet's designs despite reduced stability and suboptimal block usage, suggesting that they prioritised overall shape over both of these. As in Fig 4d, BloxNet's "Eiffel Tower" was taller but less stable, and its "burger" achieved a flatter profile using a square block for the bun. These results reveal scope for adapting IDfRA's design priorities in future work to reflect the dominant objectives of a given task – whether that be semantic resemblance, structural soundness, or accurate use of materials.

B. VLM Test for Design Recognisability

The second comparative assessment involved prompting GPT-4o with a rendered assembly image and an unordered list of objects – asking it to rank the objects based on semantic resemblance to the image. This approach, inspired by VLM-based answer scoring methods [54] and previously employed by [18], was reproduced here to enable direct comparability with BloxNet. To avoid trivialising the task with unrelated distractors, we prompted GPT-4o to generate a list of 200 random objects including and semantically related to the 15 representative assemblies. For each of the 15 assemblies (IDfRA and BloxNet), we constructed ranking trials with list lengths $N = 5, 10, 15, 20$, where each unordered list contained the correct label and $N - 1$ distractors. GPT-4o was then prompted to rank the objects by similarity to

N	Method	Top-1 Accuracy	Average Ranking	Relative Ranking
5	Iterative DfRA	**64.44%**	**1.60**	**32.00%**
	BloxNet	62.22%	1.62	32.44%
10	Iterative DfRA	**73.33%**	**2.24**	**22.44%**
	BloxNet	57.78%	2.38	23.78%
15	Iterative DfRA	**68.89%**	**2.71**	**18.07%**
	BloxNet	51.11%	3.36	22.37%
20	Iterative DfRA	**46.67%**	**4.38**	**21.89%**
	BloxNet	42.22%	5.40	27.00%

TABLE I. VLM-based design recognisability: Top-1 accuracy, average ranking, and relative ranking averaged across three runs. Computed using GPT-4o rankings of N objects chosen from a pool of 200 objects based on semantic resemblance.

Assembly Design	% Blocks Correctly Placed	% Assemblies Successful
Giraffe	97.78	80
Taj Mahal	100	100
House	100	100
Shark	95	70
Burger	100	100
Jungle	95.71	70

TABLE II. Physical feasibility analysis: Percentage of blocks correctly placed by the robot and percentage of successful assemblies across 10 trials for six assembly designs.

the rendered assembly image. Full object lists and code are included in the codebase.

As in [18], we report (1) Top-1 accuracy, the percentage of assemblies in which the correct label is ranked first, (2) Average Ranking, the mean ranking assigned to the correct label across all 15 assemblies, and (3) Relative Ranking, the mean rank of the correct label normalised by the total number of choices N in the list.

The results of this experiment are presented in Table I. IDfRA consistently outperforms BloxNet in this evaluation, obtaining up to 73.33% top-1 accuracy at $N = 10$. While IDfRA designs generally received higher rankings, both methods exhibited a common limitation: structures with similar high-level shapes were frequently confused – "sheep" and "house" were often misclassified as "table-like" structures such as "side table." Additionally, certain structures disproportionately impacted performance. For example, "shark" and "piano" reduced accuracy for both methods, whereas "sheep" particularly reduced IDfRA's scores, while "Taj Mahal" degraded BloxNet's.

C. Robot Assembly Test for Feasibility

Next, we independently assessed the physical feasibility of IDfRA's generated designs within the simulation environment. For this analysis, we chose six assemblies, as in [18], to cover a range of block types, stacking heights, and difficulty levels. For each, the generated plans were executed in simulation over 10 trials, with the percentage of correctly placed blocks and fully successful build attempts reported in Table II.

This evaluation aims to verify whether IDfRA's designs are in principle constructible, despite using a minimal simulation environment; the setup prioritises design validation over precise robotic optimisation. The minimal simulation serves as a proxy for an industrial system – particularly in providing visual inputs to the *Judge* – and offers a practical platform for proof of concept.

As shown in Table II, IDfRA's designs are physically executable by the simulated robotic arm, with three out of six structures successfully assembled in all 10 trials; the remaining achieving at least 70% success. Failures in "giraffe" were attributed to the lack of collision checks, which caused the tail block to knock over the neck despite the design itself being valid (as evidenced by perfect execution in the other trials). "Shark" and "jungle" exhibited unsuccessful trials due to isolated block placement errors – cuboids at the base edges requiring precise placement sometimes toppled during execution.

D. Iterative Improvement Evaluation

We examined how assembly designs evolved over ten iterations for each of the 15 target structures. For a given structure, each iteration was compared pairwise against all previous ones (e.g., iteration 3 vs. 2 and 1), yielding a binary outcome: 1 if the later design was better or equal, 0 otherwise. Each comparison was evaluated manually due to empirical limitations of VLMs in highly precise physical reasoning. If one design had missing blocks, it automatically lost. If neither (or both) designs had missing blocks, the more stable design was preferred. For equally stable designs, the more semantically recognisable design won.

For each iteration i, we computed a cumulative score encoding the number of preceding iterations it matched or outperformed. These scores are plotted on the Y-axis of Fig. 6 for iterations 1–9 (iteration 0 being the initial assembly). In this setup, a monotonically increasing line indicates a design that remains the same or improves over time.

Fig 6 illustrates that designs remained the same or improved across iterations on average (black dashed line) with a greater standard deviation (shaded grey) in later iterations. Drops in the coloured lines correspond to unstable or incomplete assemblies, which received lower scores. Overall, the results confirm that designs remain the same or improve via iterative refinement, albeit not always monotonically.

V. Conclusion

Experimental results demonstrate that IDfRA generates semantically recognisable and feasible assembly plans, and that iterative refinement can improve Design for Robotic Assembly (DfRA). The approach shows promise as a proof of concept, but several limitations leave scope for further exploration. The LLM-based *Position* submodule remains prone to occasional inconsistencies despite prompt-engineering improvements (Fig 7a, b), though overall performance is acceptable. Generated designs depend, to some extent, on the block set provided and are constrained by how well targets can be represented by a limited set of blocks. The simulation environment was intentionally kept minimal to serve as a proxy for real robotic systems, with recorded GIFs mimicking real-world execution videos used in the planning pipeline. This design choice enables extension to physical

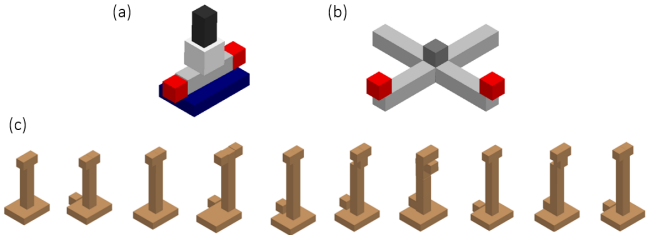

Fig. 7. Limitations. IDfRA occasionally generates designs which contain overhanging blocks or include additional unnecessary details (highlighted in red) such as for "ship" (a) and "ceiling fan" (b). Though rare, it is also possible that no iteration produces a 'good' design such as this run of "giraffe" (c).

robots and specialised manufacturing settings, where visual feedback plays a central role.

Finally, behaviour varies markedly across runs due to the inherent stochasticity of VLM generation. While this variability can produce diverse and creative "good" designs, it also introduces the rare risk that no satisfactory design emerges (Fig 7c). Moreover, iterative refinement does not always yield monotonic improvement. Future work could address this by incorporating persistent memory, such as a "common mistakes to avoid" module or skill library [24], [34], to encourage more consistent progress across iterations.

Despite these limitations, IDfRA achieves Top-1 accuracy of up to 73.33% in VLM-based object recognisability, and outperforms BloxNet in 8 out of 15 designs based on pairwise human evaluations – indicating enhanced semantic alignment. The generated plans are also physically feasible, with an overall success rate of 86.67% across six representative assemblies. Iterative evaluation further shows that designs often improve across successive refinements despite fluctuations. These findings highlight IDfRA's potential as a self-refining, foundation-model-driven system capable of generating executable designs from natural language instructions. IDfRA holds particular promise for flexible robotic assembly in underspecified or evolving contexts, as it can dynamically adapt to feedback and environmental constraints through iterative self-verification and refinement.

ACKNOWLEDGMENT

We would like to acknowledge the Engineering and Physical Sciences Research Council awards EP/V062123/1 and EP/N509620/1.

REFERENCES

[1] J. F. Engelberger and G. C. Devol, "Programmed article transfer," Patent US2 988 237A, May 13, 1954, https://patents.google.com/patent/US2988237A/en.

[2] Y. Jiang, Z. Huang, B. Yang, and W. Yang, "A review of robotic assembly strategies for the full operation procedure: planning, execution and evaluation," *Robotics and Computer-Integrated Manufacturing*, vol. 78, p. 102366, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0736584522000540

[3] A. R. Das and M. Koskinopoulou, "Toward sustainable manufacturing: A review on innovations in robotic assembly and disassembly," *IEEE Access*, vol. 13, pp. 100 149–100 166, June 2025, publisher's PDF version, copyright ©2025 The Authors.

[4] G. Boothroyd, *Assembly Automation and Product Design*, 2nd ed. CRC Press, 2005. [Online]. Available: https://www.iqytechnicalcollege.com/BAE%20682-Assembly%20Automation%20%26%20Product%20Design.pdf

[5] R. W. Kennard and L. A. Stone, "Computer aided design of experiments," *Technometrics*, vol. 11, no. 1, pp. 137–148, 1969. [Online]. Available: http://www.jstor.org/stable/1266770

[6] T.-C. Chang, R. A. Wysk, and H.-P. Wang, *Computer-Aided Manufacturing*. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1991.

[7] Z. Bi and X. Wang, *Computer Aided Design and Manufacturing*. Hoboken, NJ: John Wiley & Sons, 2020.

[8] S. Hagemann and R. Stark, "An optimal algorithm for the robotic assembly system design problem: An industrial case study," *CIRP Journal of Manufacturing Science and Technology*, 09 2020.

[9] X. Zhu, D. Jha, D. Romeres, L. Sun, M. Tomizuka, and A. Cherian, "Multi-level reasoning for robotic assembly: From sequence inference to contact selection," 05 2024, pp. 816–823.

[10] Y. Tian, K. Willis, B. Omari, J. Luo, P. Ma, Y. Li, F. Javid, E. Gu, J. Jacob, S. Sueda, H. Li, S. Chitta, and W. Matusik, "Asap: Automated sequence planning for complex robotic assembly with physical feasibility," 05 2024, pp. 4380–4386.

[11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[12] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022.

[13] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," 2022. [Online]. Available: https://arxiv.org/abs/2204.01691

[14] N. Wake, A. Kanehira, K. Sasabuchi, J. Takamatsu, and K. Ikeuchi, "Gpt-4v(ision) for robotics: Multimodal task planning from human demonstration," *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 10 567–10 574, 2024.

[15] Y. Jin, D. Li, Y. A, J. Shi, P. Hao, F. Sun, J. Zhang, and B. Fang, "Robotgpt: Robot manipulation learning from chatgpt," *IEEE Robotics and Automation Letters*, vol. PP, pp. 1–8, 03 2024.

[16] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," 2025. [Online]. Available: https://arxiv.org/abs/2504.16054

[17] H. Fan, X. Liu, J. Y. H. Fuh, Y. Zhang, and H.-C. Lee, "Embodied intelligence in manufacturing: Leveraging large language models for autonomous industrial robotics," *Journal of Intelligent Manufacturing*, vol. 36, pp. 1141–1157, 2025.

[18] A. Goldberg, K. Kondap, T. Qiu, Z. Ma, L. Fu, J. Kerr, H. Huang, K. Chen, K. Fang, and K. Goldberg, "Blox-net: Generative design-for-robot-assembly using vlm supervision, physics, simulation, and a robot with reset," in *2025 International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.

[19] K. T. Estelle and B. A. Gozen, "Precision flow rate control during micro-scale material extrusion by iterative learning of pressure-flow rate relationships," *Additive Manufacturing*, vol. 82, p. 104031, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214860424000770

[20] A. Aniculaesei, D. Arnsberger, F. Howar, and A. Rausch, "Towards the verification of safety-critical autonomous systems in dynamic environments," *Electronic Proceedings in Theoretical Computer Science*, vol. 232, pp. 79–90, 12 2016.

[21] W. Xia, R. Feng, D. Wang, and D. Hu, "Phoenix: A motion-based

self-reflection framework for fine-grained robotic action correction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 6981–6990.

[22] F. Joublin, A. Ceravola, P. Smirnov, F. Ocker, J. Deigmoeller, A. Belardinelli, C. Wang, S. Hasler, D. Tanneberg, and M. Gienger, "Copal: Corrective planning of robot actions with large language models," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2024, p. 8664–8670. [Online]. Available: http://dx.doi.org/10.1109/ICRA57147.2024.10610434

[23] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and brian ichter, "Inner monologue: Embodied reasoning through planning with language models," in *6th Annual Conference on Robot Learning*, 2022. [Online]. Available: https://openreview.net/forum?id=3R3Pz5i0tye

[24] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=ehfRiF0R3a

[25] International Federation of Robotics, "World robotics 2024: Industrial robots – executive summary," International Federation of Robotics, Tech. Rep., 2024, accessed: 2025-08-02. [Online]. Available: https://ifr.org/img/worldrobotics/Executive_Summary_WR_2024_Industrial_Robots.pdf

[26] P. Dewhurst and G. Boothroyd, "Early cost estimating in product design," *Journal of Manufacturing Systems*, vol. 7, p. 183–191, 12 1988.

[27] S. Miyakawa and T. Ohashi, "The hitachi assemblability evaluation method (aem)," in *Proceedings of the International Conference on Product Design for Assembly*, Newport, RI, USA, 1986.

[28] L. E. S. Ltd and U. of Hull, *Design For Manufacture and Assembly Practitioners Manual*, version 10 ed. Hull, UK: Lucas Engineering Systems Ltd, 1993.

[29] R. W. Kennard and L. A. Stone, "Computer aided design of experiments," *Technometrics*, vol. 11, no. 1, pp. 137–148, 1969.

[30] T.-C. Chang, R. A. Wysk, and H.-P. Wang, *Computer-aided Manufacturing*. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1991.

[31] Z. Bi and X. Wang, *Computer Aided Design and Manufacturing*. Hoboken, NJ: John Wiley & Sons, 2020.

[32] P. Kosky, R. Balmer, W. Keat, and G. Wise, "Chapter 10 - manufacturing engineering," in *Exploring Engineering (Third Edition)*, third edition ed., P. Kosky, R. Balmer, W. Keat, and G. Wise, Eds. Boston: Academic Press, 2013, pp. 205–235. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780124158917000108

[33] A. Cebulla, T. Asfour, and T. Kröger, "Speeding up assembly sequence planning through learning removability probabilities," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 12 388–12 394.

[34] Y. Guo, B. Tang, I. Akinola, D. Fox, A. Gupta, and Y. Narang, "SRSA: Skill retrieval and adaptation for robotic assembly tasks," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=RInisw1yin

[35] K. Nagpal and N. Mehr, "Optimal robotic assembly sequence planning (orasp): A sequential decision-making approach," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 9847–9854.

[36] A. Cebulla, T. Asfour, and T. Kröger, "Beyond feasibility: Efficiently planning robotic assembly sequences that minimize assembly path lengths," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 4076–4083.

[37] T.-H. Eng, Z.-K. Ling, W. Olson, and C. McLean, "Feature-based assembly modeling and sequence generation," *Computers & Industrial Engineering*, vol. 36, no. 1, pp. 17–33, 1999. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0360835298001065

[38] C. Tie, S. Sun, J. Zhu, Y. Liu, J. Guo, Y. Hu, H. Chen, J. Chen, R. Wu, and L. Shao, "Manual2skill: Learning to read manuals and acquire robotic skills for furniture assembly using vision-language models," in *The first CVPR workshop on 3D Vision Language Models (VLMs) for Robotics Manipulation: Opportunities and Challenges*, 2025. [Online]. Available: https://openreview.net/forum?id=K3JtdYXT6J

[39] A. C. Doris, D. Grandi, R. Tomich, M. F. Alam, M. Ataei, H. Cheong, and F. Ahmed, "Designqa: A multimodal benchmark for evaluating large language models' understanding of engineering documentation," *Journal of Computing and Information Science in Engineering*, vol. 25, no. 2, p. 021009, 12 2024. [Online]. Available: https://doi.org/10.1115/1.4067333

[40] S. Wu, A. Khasahmadi, M. Katz, P. K. Jayaraman, Y. Pu, K. Willis, and B. Liu, "CAD-LLM: Large Language Model for CAD Generation," in *NeurIPS 2023 Workshop on Machine Learning for Creativity and Design*, 2023, outstanding Paper Award.

[41] H. You, Y. Ye, T. Zhou, Q. Zhu, and J. Du, "Robot-enabled construction assembly with automated sequence planning based on chatgpt: Robogpt," *Buildings*, vol. 13, no. 7, 2023. [Online]. Available: https://www.mdpi.com/2075-5309/13/7/1772

[42] A. Macaluso, N. Cote, and S. Chitta, "Toward Automated Programming for Robotic Assembly Using ChatGPT," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 17 687–17 693.

[43] P. Huang, X. Zhang, Z. Cao, S. Liu, M. Xu, W. Ding, J. Francis, B. Chen, and D. Zhao, "What went wrong? closing the sim-to-real gap via differentiable causal discovery," in *7th Annual Conference on Robot Learning*, 2023. [Online]. Available: https://openreview.net/forum?id=zvl2LuLTtgr

[44] B. Tang, M. Lin, I. Akinola, A. Handa, G. Sukhatme, F. Ramos, D. Fox, and Y. Narang, "Industreal: Transferring contact-rich assembly tasks from simulation to reality," 07 2023.

[45] G. J. Gao, T. Li, J. Shi, Y. Li, Z. Zhang, N. Figueroa, and D. Jayaraman, "VLMgineer: Vision language models as robotic toolsmiths," in *1st Workshop on Robot Hardware-Aware Intelligence*, 2025. [Online]. Available: https://openreview.net/forum?id=i3JNInaLb9

[46] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. P. Saldyt, and A. B. Murthy, "Position: LLMs can't plan, but can help planning in LLM-modulo frameworks," in *Forty-first International Conference on Machine Learning*, 2024. [Online]. Available: https://openreview.net/forum?id=Th8JPEmH4z

[47] Y. Jin, D. Li, Y. A, J. Shi, P. Hao, F. Sun, J. Zhang, and B. Fang, "Robotgpt: Robot manipulation learning from chatgpt," *IEEE Robotics and Automation Letters*, vol. PP, pp. 1–8, 03 2024.

[48] K. Liang, Z. Zhang, and J. F. Fisac, "Introspective planning: Aligning robots' uncertainty with inherent task ambiguity," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://openreview.net/forum?id=4TlUE0ufiz

[49] B. Li, P. Wu, P. Abbeel, and J. Malik, "Interactive task planning with language models," *Transactions on Machine Learning Research*, 2025. [Online]. Available: https://openreview.net/forum?id=VmfWywWuYQ

[50] P. Liu, Y. Orru, J. Vakil, C. Paxton, N. Shafiullah, and L. Pinto, "Demonstrating ok-robot: What really matters in integrating open-knowledge models for robotics," 07 2024.

[51] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, and B. Zitkovich, "Rt-1: Robotics transformer for real-world control at scale," 07 2023.

[52] D. I. Campbell, S. Rane, T. Giallanza, C. N. D. Sabbata, K. Ghods, A. Joshi, A. Ku, S. M. Frankland, T. L. Griffiths, J. D. Cohen, and T. W. Webb, "Understanding the limits of vision language models through the lens of the binding problem," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://openreview.net/forum?id=Q5RYn6jagC

[53] OpenAI, "Introducing structured Outputs in the API," OpenAI Developer Documentation, Aug. 2024, structured Outputs feature for enforced JSON-schema compliance via chat completion API. [Online]. Available: https://platform.openai.com/docs/guides/structured-outputs

[54] L. Fu, G. Datta, H. Huang, W. C.-H. Panitch, J. Drake, J. Ortiz, M. Mukadam, M. Lambeta, R. Calandra, and K. Goldberg, "A touch, vision, and language dataset for multimodal alignment," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.